

实验报告一

赵莘萌

8.30

目录

1	课上练习	1
1.1	第一题	1
1.2	第二题	5
1.3	第三题	7
1.4	第四题	9
2	课后习题	10
2.1	ls 的 -l 选项 (flag) 作用是什么? 运行 ls -l / 并观察输出。每一行最前面的 10 个字符分别代表什么? (提示: man ls) . . .	10
2.2	'单引号'、"双引号"和 'ANSI '、! 和换行符的字符串。	11
2.3	为什么 cd 必须是 Shell 内建命令, 而不能是独立程序? (提示: 想想子进程能影响和不能影响父进程的哪些状态。	12
2.4	在命令 find /Downloads -type f -name "*.zip" -mtime +30 中, *.zip 是一个「glob」。什么是 glob? 新建一个测试目录并创建一些文件, 试试 ls *.txt、ls file?.txt、ls a,b,c.txt 等模式。 . . .	13
2.5	Shell 有三条标准流: stdin (0)、stdout (1)、stderr (2)。运行 'ls /nonexistent /tmp', 把 stdout 和 stderr 分别重定向到两个文件。你将如何把两者都重定向到同一个文件? 参见 Redirections。	14

2.6	\$? 保存上一条命令的退出状态 (0 表示成功)。仅在前一条成功时执行后一条; 仅在前一条失败时执行后一条。写一个一行命令: 仅当 /tmp/mydir 不存在时才创建它。参见 Exit Status。	14
2.7	写一个脚本,接收文件名参数(\$1),用 test -f 或 [-f...] 检查该文件是否存在,并根据结果输出不同提示。参见 BashConditional Expressions。	15
2.8	把上一题完成的脚本保存为文件(如 check.sh)。先运行 ./check.sh somefile, 会发生什么? 然后执行 chmod +x check.sh 再试一次。为什么这一步是必须的? (提示: 比较 chmod 前后的 ls -l check.sh 输出)	15
2.9	在脚本的 set 选项 (flag) 里加入 -x 会发生什么? 写个简单脚本试试并观察输出。参见 The Set Builtin。	15
2.10	在脚本的 set 选项 (flag) 里加入 -x 会发生什么? 写个简单脚本试试并观察输出。参见 The Set Builtin。	16
3	解题感悟	16

1 课上练习

1.1 第一题

```
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179
zxm@zxm-VirtualBox: ~/桌面$ cd ..
zxm@zxm-VirtualBox: $ cd desktop
bash: cd: desktop: 没有那个文件或目录
zxm@zxm-VirtualBox: $ cd 桌面
zxm@zxm-VirtualBox: ~/桌面$ mkdir q01
zxm@zxm-VirtualBox: ~/桌面$ cd q01
zxm@zxm-VirtualBox: ~/桌面/q01$ mkdir -p input/docs input/tmp
zxm@zxm-VirtualBox: ~/桌面/q01$ cat > "input/docs/notes one.txt" <<EOF
> alpha
> beta
> EOF
zxm@zxm-VirtualBox: ~/桌面/q01$ echo "hidden" > input/docs/.secret.txt
zxm@zxm-VirtualBox: ~/桌面/q01$ touch input/tmp/empty.txt
zxm@zxm-VirtualBox: ~/桌面/q01$ cat > input/run.log << EOF
> start task
> finish task
> EOF
zxm@zxm-VirtualBox: ~/桌面/q01$ pwd
/home/zxm/桌面/q01
zxm@zxm-VirtualBox: ~/桌面/q01$ ls -la input
总计 12
drwxrwxr-x 2 zxm zxm 4096 8月 20 14:04 docs
-rw-rw-r-- 1 zxm zxm  23 8月 20 14:06 run.log
drwxrwxr-x 2 zxm zxm 4096 8月 20 14:04 tmp
zxm@zxm-VirtualBox: ~/桌面/q01$ mkdir -q work/25020007179
mkdir: 无效的选项 -- q
请尝试执行 "mkdir --help" 来获取更多信息。
zxm@zxm-VirtualBox: ~/桌面/q01$ mkdir -p work/25020007179
```

```
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179
zxm@zxm-VirtualBox: ~/桌面/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007179/ \;
find: 缺少 "--exec"参数
zxm@zxm-VirtualBox: ~/桌面/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/20250007179/ \;
cp: 使用 --parents 时，目标必须是目录
请尝试执行 "cp --help" 来获取更多信息。
cp: 使用 --parents 时，目标必须是目录
请尝试执行 "cp --help" 来获取更多信息。
cp: 使用 --parents 时，目标必须是目录
请尝试执行 "cp --help" 来获取更多信息。
zxm@zxm-VirtualBox: ~/桌面/q01$ find input -type f -name "*.txt" -exec cp --parents {} work/25020007179/ \;
zxm@zxm-VirtualBox: ~/桌面/q01$ cd work/25020007179
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ find . -type d -exec chmod 750 {} \;
find: 'type': 没有那个文件或目录
find: 'd': 没有那个文件或目录
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ ^C
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ find . -type d -exec chmod 750 {} \;
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ find . -type f -exec chmod 640 {} \;
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ find . -type f | while read -r file; do
> size=$(wc -c < "$file")
> echo "$file $size"
> done > inventory.txt
zxm@zxm-VirtualBox: ~/桌面/q01/work/25020007179$ ls -R
.:
input  inventory.txt
./input:
docs  tmp

./input/docs:
'notes one.txt'

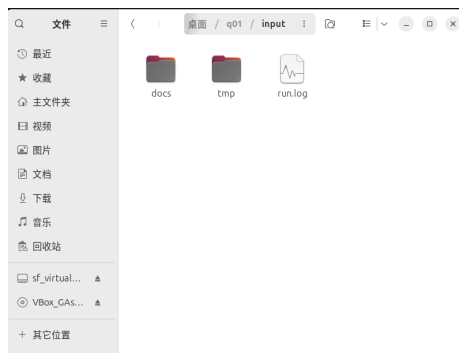
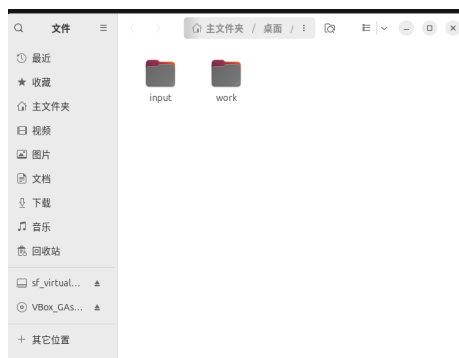
./input/tmp:
empty.txt
```

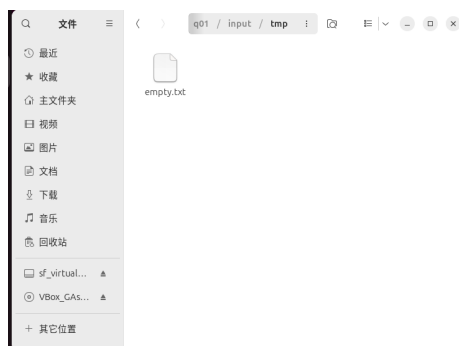
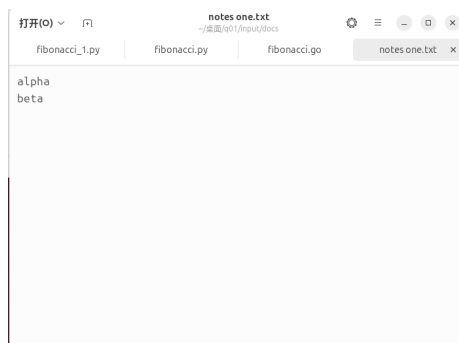
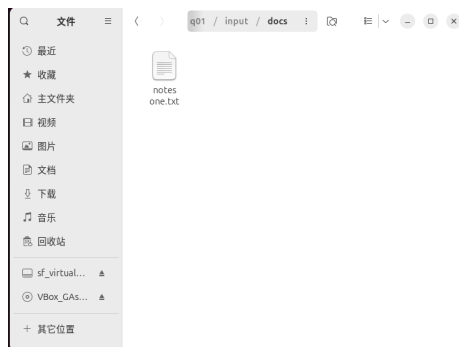
```
szm@xzn-VirtualBox:~/桌面/q01/work/250200071705$ ls -lR
.:
总计 8
drwxr-xr-x 4 xzn xzn 4096 8月 28 14:15 input
-rw-rw-r-- 1 xzn xzn 99 8月 28 14:23 inventory.txt

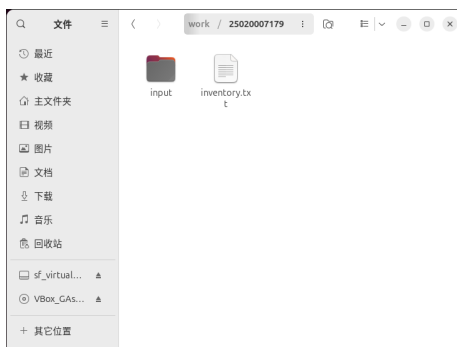
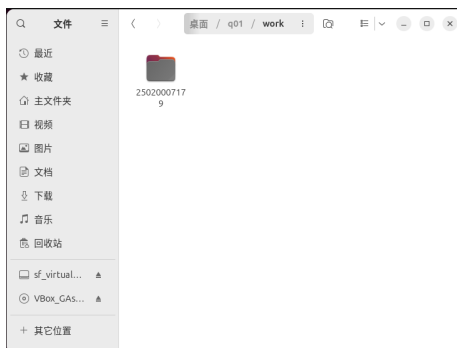
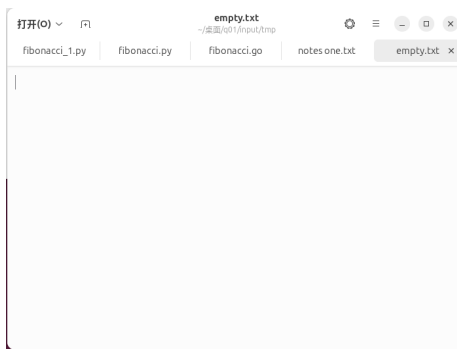
./input:
总计 0
drwxr-xr-x 2 xzn xzn 4096 8月 28 14:15 docs
drwxr-xr-x 2 xzn xzn 4096 8月 28 14:15 tmp

./input/docs:
总计 4
-rw-r----- 1 xzn xzn 11 8月 28 14:15 'notes one.txt'

./input/tmp:
总计 0
-rw-r----- 1 xzn xzn 0 8月 28 14:15 empty.txt
szm@xzn-VirtualBox:~/桌面/q01/work/250200071705$ cat inventory.txt
./inventory.txt 0
./input/tmp/empty.txt 0
./input/docs/.secret.txt 7
./input/docs/notes one.txt 11
szm@xzn-VirtualBox:~/桌面/q01/work/250200071705$ chmod 640 inventory.txt
szm@xzn-VirtualBox:~/桌面/q01/work/250200071705$
```







```
inventory.txt
~/桌面/q01/work/23020007179

py | fibonacci.py | fibonacci.go | notes one.txt | empty.txt | inventory.txt x

./inventory.txt 0
./input/tmp/empty.txt 0
./input/docs/.secret.txt 7
./input/docs/notes one.txt 11
```

1.2 第二题

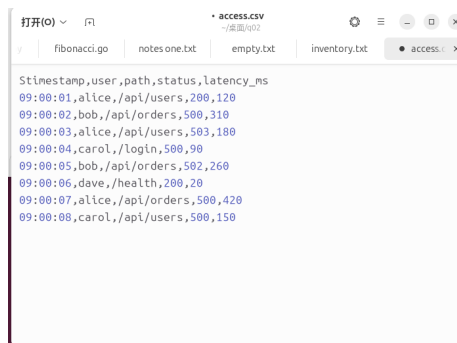
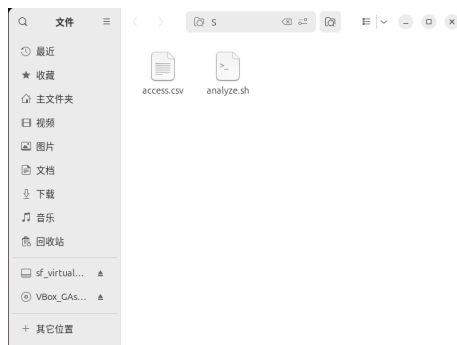
```
zxm@zxm-VirtualBox: ~/桌面/q02
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,200
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,400
09:00:08,carol,/api/users,500,150
EOF
zxm@zxm-VirtualBox: ~/桌面/q02$ cat > analyze.sh <<EOF
#!/bin/bash
SCSV=$1

if [ ! -f "$SCSV" ]; then
    echo "错误: 文件 $SCSV 不存在" >&2
    exit 1
fi

echo "Top 2 Sxx paths:"
awk -F, 'NR>1 && $4>500 && $4<600 {cnt[$3]++}
        END {for (p in cnt) print cnt[p], p}' "$SCSV" \
    | sort -k1,1nr -k2,2 \
    | head -2

echo "Average latency_ms:"
awk -F, 'NR>1 {sum+=$5; n++}
        END {printf "%.2f\n", sum/n}' "$SCSV"
EOF

zxm@zxm-VirtualBox: ~/桌面/q02$ bash analyze.sh access.csv
echo "..."
bash analyze.sh no_such_file.csv
echo "退出码: $?"
Top 2 Sxx paths:
2 /api/orders
2 /api/users
Average latency_ms:
193.75
---
错误: 文件 no_such_file.csv 不存在
退出码: 1
```




```
#!/bin/bash
CSV="$1"

if [ ! -f "$CSV" ]; then
    echo "错误：文件 $CSV 不存在" >&2
    exit 1
fi

echo "Top 2 5xx paths:"
awk -F, 'NR>1 && $4>=500 && $4<600 {cnt[$3]++}
        END {for (p in cnt) print cnt[p], p}' "$CSV" \
| sort -k1,1nr -k2,2 \
| head -2

echo "Average latency_ms:"
awk -F, 'NR>1 {sum+=$5; n++}
        END {printf "%.2f\n", sum/n}' "$CSV"

~
~
~
~
~/桌面/q02/analyze.sh 17L, 375B 1,1 全部
```

1.3 第三题

```
zxm@zxm-VirtualBox:~/桌面$ mkdir q03
zxm@zxm-VirtualBox:~/桌面$ cd q03
zxm@zxm-VirtualBox:~/桌面/q03$ git init
找不到命令 "git"，您可以通过以下软件包安装它：
sudo apt install git
zxm@zxm-VirtualBox:~/桌面/q03$ sudo apt install git
[sudo] zxm 的密码：
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
```

```

xzm@xzm-VirtualBox: ~/桌面/q03
正在设置 git (1:2.43.0-1ubuntu7.3) ...
正在处理用于 nan-db (2.12.0-4build2) 的触发器 ...
xzm@xzm-VirtualBox:~/桌面/q03$ git init
提示: 使用 "master" 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示: 配置使用初始分支名, 并清除这条警告, 请执行:
提示:  git config --global init.defaultBranch <名称>
提示:
提示: 除了 "master" 之外, 通常选定的名字有 "main", "trunk" 和 "development".
提示: 可以通过以下命令重命名刚创建的分支:
提示:  git branch -m <name>
已初始化的 Git 仓库于 /home/xzm/桌面/q03/.git/
xzm@xzm-VirtualBox:~/桌面/q03$ git config --global init.defaultBranch main
xzm@xzm-VirtualBox:~/桌面/q03$ git branch
xzm@xzm-VirtualBox:~/桌面/q03$ git branch -m main
xzm@xzm-VirtualBox:~/桌面/q03$ git branch
xzm@xzm-VirtualBox:~/桌面/q03$ echo "mode:normal" > config.txt
xzm@xzm-VirtualBox:~/桌面/q03$ git add config.txt && git commit -m "init"
作者身份未知

*** 请告诉我你是谁。

运行

git config --global user.email "you@example.com"
git config --global user.name "Your Name"

来设置您账号的缺省身份标识。
如果仅在本仓库设置身份标识, 则省略 --global 参数。

fatal: 无法自动探测邮件地址 (得到 'xzm@xzm-VirtualBox.(none)')
xzm@xzm-VirtualBox:~/桌面/q03$ ^C
xzm@xzm-VirtualBox:~/桌面/q03$ git config --global user.email "xm.zhao@outlook.com"
xzm@xzm-VirtualBox:~/桌面/q03$ git config --global user.name "xzm"

```

```

xzm@xzm-VirtualBox:~/桌面/q03$ git add config.txt && git commit -m "init"
(main (根提交) 6418d55) init
1 file changed, 1 insertion(+)
create mode 100644 config.txt
xzm@xzm-VirtualBox:~/桌面/q03$ git checkout -b feature-a
切换到一个新分支 'feature-a'
xzm@xzm-VirtualBox:~/桌面/q03$ echo "mode:safe" > config.txt
xzm@xzm-VirtualBox:~/桌面/q03$ git add . && git commit -m "safe"
(feature-a 317911a) safe
1 file changed, 1 insertion(+), 1 deletion(-)
xzm@xzm-VirtualBox:~/桌面/q03$ git checkout main
切换到分支 'main'
xzm@xzm-VirtualBox:~/桌面/q03$ git checkout -b feature-b
切换到一个新分支 'feature-b'
xzm@xzm-VirtualBox:~/桌面/q03$ echo "mode:fast" > config.txt
xzm@xzm-VirtualBox:~/桌面/q03$ git add . && git commit -m "fast"
(feature-b 302ae23) fast
1 file changed, 1 insertion(+), 1 deletion(-)
xzm@xzm-VirtualBox:~/桌面/q03$ git checkout main
切换到分支 'main'
xzm@xzm-VirtualBox:~/桌面/q03$ git merge feature-a
更新 6418d55..317911a
Fast-forward
 config.txt | 2 ++
1 file changed, 1 insertion(+), 1 deletion(-)
xzm@xzm-VirtualBox:~/桌面/q03$ git merge feature-b
自动合并 config.txt
冲突 (内容): 合并冲突于 config.txt
自动合并失败, 修正冲突然后提交修正的结果。
xzm@xzm-VirtualBox:~/桌面/q03$ git add config.txt && git commit -m "resolve"
(main dccb160) resolve

```

```

xzm@xzm-VirtualBox:~/桌面/q03$ git status
位于分支 main
无文件要提交, 干净的工作区
xzm@xzm-VirtualBox:~/桌面/q03$ git log --all --graph --oneline
* dccb160 (HEAD -> main) resolve
|
* 302ae23 (feature-b) fast
* | 317911a (feature-a) safe
|/
* 6418d55 init

```

1.4 第四题

```
zxm@zxm-VirtualBox:~/桌面$ cd q04
zxm@zxm-VirtualBox:~/桌面/q04$ nano report.tex
zxm@zxm-VirtualBox:~/桌面/q04$ latexmk -pdf -halt-on-error report.tex
```

```
%
\documentclass{article}
\usepackage{amsmath}

\title{Tool Report}
\author{zhaoxinmeng-25020007179}
\date{\today}

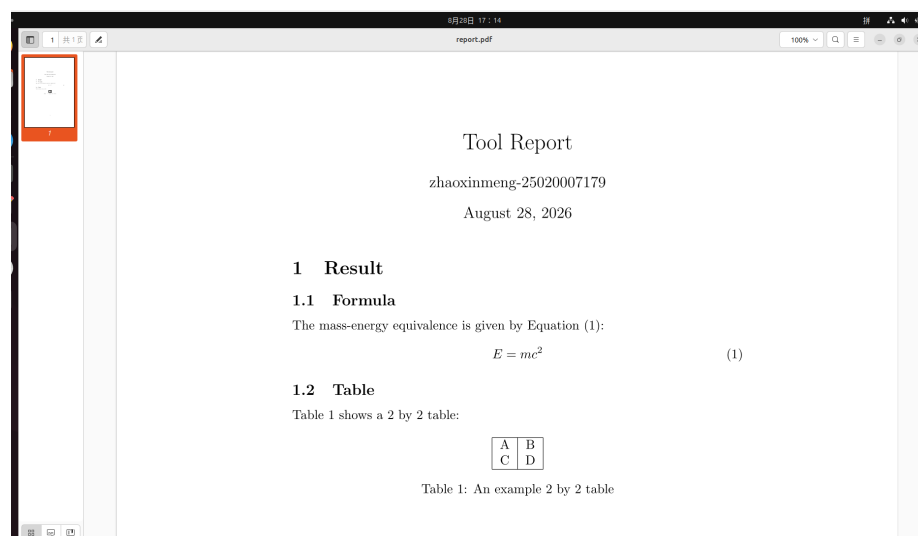
\begin{document}
\maketitle

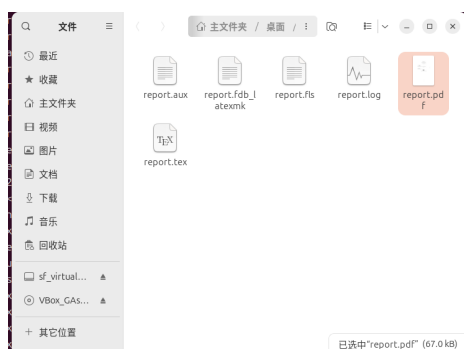
\section{Result}

\subsection{Formula}
The mass-energy equivalence is given by Equation-\eqref{eq:energy}:
\begin{equation}
E = mc^2 \label{eq:energy}
\end{equation}

\subsection{Table}
Table-\ref{tab:example} shows a 2 by 2 table:
\begin{table}[h]
\caption{Table 1}
\begin{tbl_struct}
\newtbl {tbl_struct}
\newtblrow {tbl_struct}{1}{1}{A}{B}
\newtblrow {tbl_struct}{1}{2}{C}{D}
\endtbl_struct
\begin{tbl_info cols=2}\tbl_r1{1}{1}{A}{B}\tbl_r2{1}{2}{C}{D}
\begin{tbl_r}{1}{1}{A}{B}
\end{tbl_r}
\begin{tbl_r}{1}{2}{C}{D}
\end{tbl_r}
\end{tbl_info}
\begin{tbl_r}{1}{1}{A}{B}
\end{tbl_r}
\begin{tbl_r}{1}{2}{C}{D}
\end{tbl_r}
\end{table}
\caption{An example 2 by 2 table}
\label{tab:example}
\end{table}

\end{document}
```





2 课后习题

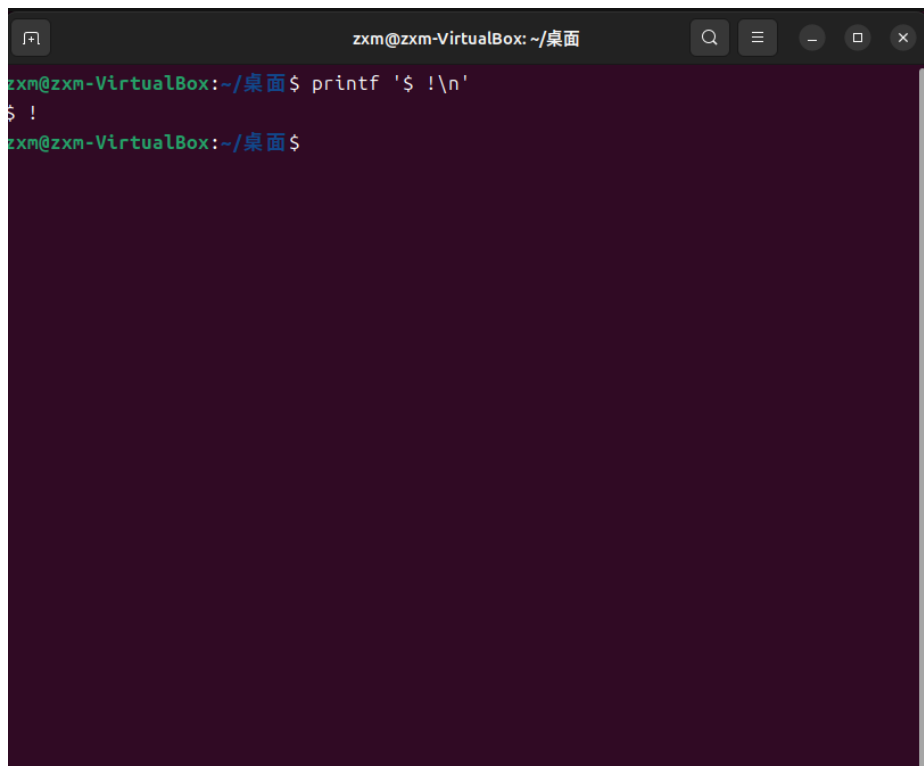
2.1 `ls` 的 `-l` 选项 (flag) 作用是什么? 运行 `ls -l /` 并观察输出。 每一行最前面的 10 个字符分别代表什么? (提示: `man ls`)

`ls -l` 以长格式列出文件, 显示权限、链接数、所有者、所属组、大小、修改时间和文件名。每一行最前面 10 个字符: 第 1 位是文件类型 (- 普通文件、d 目录、l 符号链接、c 字符设备、b 块设备); 后 9 位是权限位, 分 3 组, 每组 3 位, 依次为所有者 (u)、同组用户 (g)、其他人 (o) 的读 (r)、写 (w)、执行 (x) 权限, 无该权限用 - 表示。例如 `-rw-r--r--` 表示普通文件, 所有者可读可写, 同组和其他人只读。

```
zxm@zxm-VirtualBox: ~/桌面
总计 4009052
lrwxrwxrwx   1 root root      7 4月 22 2024 bin -> usr/bin
drwxr-xr-x   2 root root  4096 2月 26 2024 bin.usr-is-merged
drwxr-xr-x   3 root root  4096 10月 20 2025 boot
dr-xr-xr-x   2 root root  4096 8月 6 2025 cdrom
drwxr-xr-x  19 root root  4240 8月 30 17:58 dev
drwxr-xr-x 144 root root 12288 8月 28 16:56 etc
drwxr-xr-x   3 root root  4096 10月 19 2025 home
lrwxrwxrwx   1 root root      7 4月 22 2024 lib -> usr/lib
lrwxrwxrwx   1 root root      9 4月 22 2024 lib64 -> usr/lib64
drwxr-xr-x   2 root root  4096 4月 8 2024 lib.usr-is-merged
drwx-----  2 root root 16384 10月 19 2025 lost+found
drwxr-xr-x   6 root root  4096 11月 3 2025 media
drwxr-xr-x   2 root root  4096 8月 6 2025 mnt
drwxr-xr-x   3 root root  4096 10月 19 2025 opt
dr-xr-xr-x 294 root root      0 8月 30 17:58 proc
drwx-----  5 root root  4096 11月 5 2025 root
drwxr-xr-x  35 root root   900 8月 30 18:00 run
lrwxrwxrwx   1 root root      8 4月 22 2024 sbin -> usr/sbin
drwxr-xr-x   2 root root  4096 3月 31 2024 sbin.usr-is-merged
drwxr-xr-x  15 root root  4096 8月 28 13:47 snap
drwxr-xr-x   2 root root  4096 8月 6 2025 srv
-rw-----  1 root root 4105175040 10月 19 2025 swap.img
```

2.2 ‘单引号’、”双引号”和‘ANSI’、!和换行符的字符串。

单引号'...'内所有字符均为字面量，不展开变量、不解析转义；双引号"..."内会展开变量（var）和命令替换，但!在交互 shell 中可能触发历史扩展；\$'...'（ANSI-C 引号）支持 \n、\t 等转义序列，同时为字面量。输出同时包含字面 \$、! 和换行符的命令：printf '\$!\n'

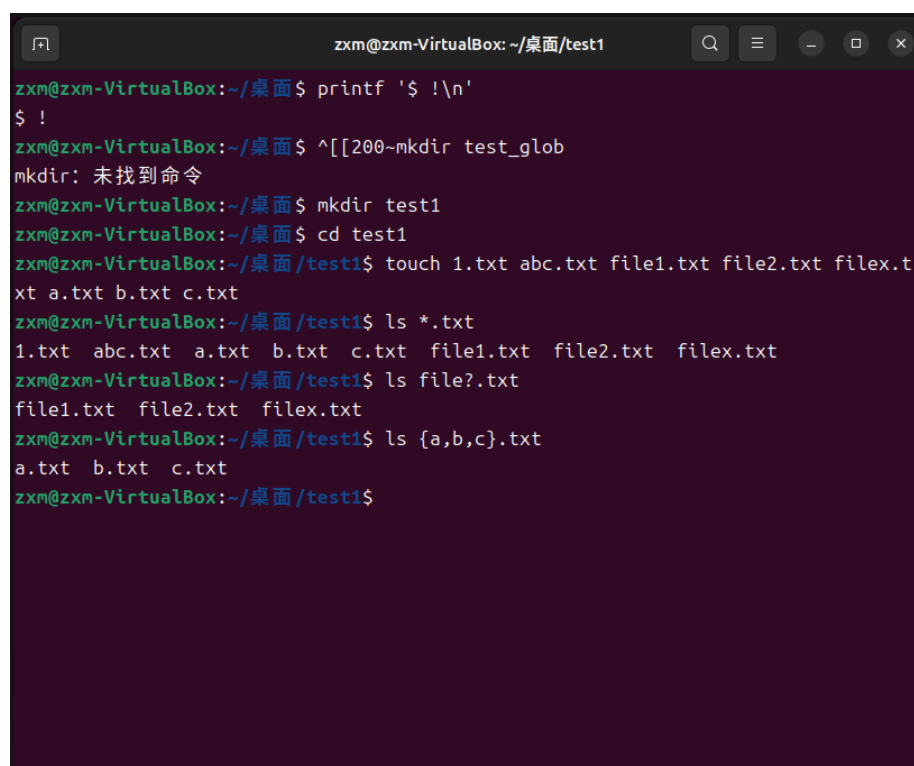
A terminal window titled 'zxm@zxm-VirtualBox: ~/桌面' with standard window controls. The prompt is 'zxm@zxm-VirtualBox:~/桌面\$'. The user enters 'printf '\$!\n'' and the output is '\$!' on the next line. The prompt returns to 'zxm@zxm-VirtualBox:~/桌面\$'.

2.3 为什么 cd 必须是 Shell 内建命令，而不能是独立程序？(提示：想想子进程能影响和不能影响父进程的哪些状态。

cd 的作用是修改当前 Shell 进程的工作目录。如果 cd 是独立可执行程序，shell 执行它时会 fork() 创建一个子进程。子进程拥有自己独立的进程状态，子进程调用 chdir() 只能修改自己的工作目录，无法修改父 shell 进程的当前工作目录；子进程退出之后，父 shell 环境完全没有变化，达不到切换目录的效果。只有作为 Shell 内建命令，cd 直接在父 Shell 进程内部执行 chdir() 系统调用，直接修改 shell 本身的工作目录，才能真正实现目录切换。

2.4 在命令 `find /Downloads -type f -name "*.zip" -mtime +30` 中, *.zip 是一个「glob」。什么是 glob? 新建一个测试目录并创建一些文件, 试试 `ls *.txt`、`ls file?.txt`、`ls a,b,c.txt` 等模式。

glob (文件名通配): Bash shell 的文件名模式匹配机制, 使用特殊元字符 ‘*’、‘?’、‘[]’ 对文件名做模糊匹配, shell 在把参数传给程序 ** 之前 **, 会把通配模式展开成实际匹配到的文件名列表。



```
zxm@zxm-VirtualBox: ~/桌面/test1
zxm@zxm-VirtualBox:~/桌面$ printf '$ !\n'
$ !
zxm@zxm-VirtualBox:~/桌面$ ^[[200~mkdir test_glob
mkdir: 未找到命令
zxm@zxm-VirtualBox:~/桌面$ mkdir test1
zxm@zxm-VirtualBox:~/桌面$ cd test1
zxm@zxm-VirtualBox:~/桌面/test1$ touch 1.txt abc.txt file1.txt file2.txt filex.txt
zxm@zxm-VirtualBox:~/桌面/test1$ ls *.txt
1.txt abc.txt a.txt b.txt c.txt file1.txt file2.txt filex.txt
zxm@zxm-VirtualBox:~/桌面/test1$ ls file?.txt
file1.txt file2.txt filex.txt
zxm@zxm-VirtualBox:~/桌面/test1$ ls {a,b,c}.txt
a.txt b.txt c.txt
zxm@zxm-VirtualBox:~/桌面/test1$
```

2.5 Shell 有三条标准流：stdin (0)、stdout (1)、stderr (2)。
运行 ‘ls /nonexistent /tmp’，把 stdout 和 stderr 分别重定向到两个文件。你将如何把两者都重定向到同一个文件？参见 Redirections。

Shell 三条标准流：stdin（文件描述符 0）：标准输入；stdout（文件描述符 1）：标准输出，程序正常打印信息；stderr（文件描述符 2）：标准错误输出，保存报错信息。

```
zxm@zxm-VirtualBox:~/桌面/test1$ ^[[200~ls /nonexistent /tmp > out.txt 2> err.t
t
zxm@zxm-VirtualBox:~/桌面/test1$ ~ls /nonexistent /tmp > out.txt 2> err.txt
zxm@zxm-VirtualBox:~/桌面/test1$ ls /nonexistent /tmp > all.txt 2>&1
、zxm@zxm-VirtualBox:~/桌面/test1$
```

2.6 \$? 保存上一条命令的退出状态（0 表示成功）。仅在前一条成功时执行后一条；|| 仅在前一条失败时执行后一条。写一个一行命令：仅当 /tmp/mydir 不存在时才创建它。参见 Exit Status。

```
test ! -d /tmp/mydir || mkdir /tmp/mydir
```


2.7 写一个脚本，接收文件名参数 (\$1)，用 `test -f` 或 `[-f ...]` 检查该文件是否存在，并根据结果输出不同提示。参见 `Bash-Conditional Expressions`。

```
zxm@zxm-VirtualBox:~/桌面/test1$ nano checkfile.sh
zxm@zxm-VirtualBox:~/桌面/test1$ chmod +x checkfile.sh
zxm@zxm-VirtualBox:~/桌面/test1$ ./checkfile.sh out.txt
文件 out.txt 存在
zxm@zxm-VirtualBox:~/桌面/test1$ ./checkfile.sh abc.txt
文件 abc.txt 存在
zxm@zxm-VirtualBox:~/桌面/test1$ ^[[200~./checkfile.sh abc.txt
bash: ./checkfile.sh: 没有那个文件或目录
zxm@zxm-VirtualBox:~/桌面/test1$ ~./checkfile.sh abcd.txt
bash: ~./checkfile.sh: 没有那个文件或目录
zxm@zxm-VirtualBox:~/桌面/test1$
```

2.8 把上一题完成的脚本保存为文件(如 `check.sh`)。先运行 `./check.sh somefile`，会发生什么？然后执行 `chmod +x check.sh` 再试一次。为什么这一步是必须的？(提示：比较 `chmod` 前后的 `ls -l check.sh` 输出)

未执行 `chmod +x` 时运行 `./check.sh somefile`，终端提示权限不够，脚本无法运行。执行 `chmod +x check.sh` 添加执行权限后，再次运行 `./check.sh somefile`，脚本正常执行，输出文件 `somefile` 不存在。`chmod +x` 的作用是给文件添加执行权限 `x`，Linux 中使用 `./` 文件名方式运行文件时，系统要求文件具备执行权限才可以将它作为程序启动。通过 `ls -l check.sh` 可以观察，`chmod` 之前权限位没有 `x` 标记，`chmod` 之后权限位出现 `x` 执行标记，因此该步骤是必须的。

2.9 在脚本的 `set` 选项 (flag) 里加入 `-x` 会发生什么？写个简单脚本试试并观察输出。参见 `The Set Builtin`。

`set -x` 开启脚本调试模式，脚本运行时会在执行每一条命令前把该命令打印输出，输出行以 `+` 作为标记，之后输出命令执行结果，便于追踪脚本运

行过程、排查错误。测试脚本内容为!/bin/bash; set -x; a=10; echo \$a, 运行后输出 + a=10、+ echo 10、10, 带加号的内容就是 set-x 打印出的待执行命令。

```
zxm@zxm-VirtualBox:~/桌面/test1$ nano testset.sh
zxm@zxm-VirtualBox:~/桌面/test1$ chmod +x testset.sh
zxm@zxm-VirtualBox:~/桌面/test1$ ./testset.sh
+ a=10
+ echo 10
10
zxm@zxm-VirtualBox:~/桌面/test1$
```

2.10 在脚本的 set 选项 (flag) 里加入 -x 会发生什么? 写个简单脚本试试并观察输出。参见 The Set Builtin。

```
zxm@zxm-VirtualBox:~/桌面/test1$ echo "test content" > notes.txt
zxm@zxm-VirtualBox:~/桌面/test1$ cp notes.txt notes_$(date +%Y-%m-%d).txt
zxm@zxm-VirtualBox:~/桌面/test1$ ls
1.txt  a.txt  c.txt  file2.txt  notes.txt
abc.txt b.txt  err.txt filex.txt  out.txt
all.txt checkfile.sh file1.txt notes_2026-08-30.txt testset.sh
```

3 解题感悟

通过这部分 Linux Shell 练习, 我熟悉了基础命令、脚本简单编写以及调试的方法。一开始对命令替换、set-x 调试模式不太理解, 动手实操之后才弄懂原理。很多命令光看文字不好理解, 亲自敲一遍观察输出, 更容易记住用法。写备份复制这类命令时, 要注意语法格式, 小符号写错就会报错。后续还需要多动手练习, 加深对 Shell 脚本的理解。